

ללמוד Angular על ידי בניית Compass

מסלול מבוסס-פרויקט: מהשורה הראשונה שלך
ב-TypeScript ועד לאפליקציית Angular פרוסה

מהדורה ראשונה, 2026

הוכן לשימוש אישי.

הקדמה

אם מעולם לא בנית אפליקציית ווב, הספר הזה נכתב עבורך. הוא מניח שאתה יודע לקרוא אנגלית, להשתמש במחשב, ומוכן להקליד דברים בתוך עורך קוד ולראות מה קורה. הוא לא מניח שאתה יודע מהו פריימוורק, מהו TypeScript, או למה מישו יבחר מרצון להשתמש במשהו שנקרא "הזרקת תלויות". עד לפרק האחרון, כל המונחים האלה ירגישו לך ככלים שאתה שולח אליהם יד, לא כז'רגון שאתה סובל.

הספר נבנה סביב אפליקציה יחידה בשם **Compass** – מעקב אישי אחר משימות והרגלים. בפרק 4 תקליד `ng new compass` ותראה תבנית התחלתית ריקה מופיעה על המסך. כל פרק שיבוא אחר כך מוסיף יכולת אמיתית אחת ל-Compass. בסוף הספר, תהיה לך אפליקציה אמיתית: היא תאפשר לך להתחבר, לתכנן את השבוע שלך, לעקוב אחרי רצפים, לעבוד ללא חיבור לאינטרנט, לסנכרן כשתתחבר מחדש, ולפעול בדפדפן באינטרנט הפומבי.

ללמידה על ידי בנייה יש שני יתרונות שספרי "קריאה תחילה" בדרך כלל מוותרים עליהם. הראשון הוא שכל מושג נוחת איפשהו. כשאתה פוגש רעיון חדש – נניח, פיירט (interceptor) – אתה לא סתם קורא הגדרה; אתה כותב אחד, רואה איך הוא משנה את התנהגות האפליקציה שלך, ואז שובר אותו בכוונה כדי לראות מה

קורה. השני הוא שאתה מסיים עם משהו. Compass קטן מספיק כדי להחזיק בראש, ועשיר מספיק כדי להראות לחבר. הוא גם שלך: אין בספר חידה שהתשובה שלה חבויה בסוף.

איך הספר מאורגן

לספר יש חמישה חלקים ונספחים.

חלק ראשון – יסודות. ארבעה פרקים שמעבירים אותך מאפס לפרויקט Angular רץ. תפגוש את TypeScript, את התכונות המודרניות של JavaScript ש-Angular באמת משתמש בהן, ואת ה-CLI של Angular. לא תיגע כאן בקומפוננטה; המטרה היא שיהיה לך סדנה לפני שתרים פטיש.

חלק שני – קומפוננטות ותבניות. המכניקה המרכזית: מהי קומפוננטה, איך תבניות וקישורי נתונים עובדים, ואיך סיגנלים – הפרימיטיב הריאקטיבי המודרני של Angular – מאפשרים לך לתאר מצב משתנה ללא טקסים. בסוף חלק שני, Compass ייראה וירגיש כמו אפליקציה אמיתית.

חלק שלישי – שירותים והעולם שבחוץ. אפליקציות לא חיות בקומפוננטה אחת. חלק זה מכסה הזרקת תלויות, HTTP, RxJS וטפסים – ארבעת הדברים שאתה צריך כדי לדבר עם שרתים, לאסוף קלט ממשתמשים, ולשתף התנהגות בין חלקי האפליקציה בלי להעתיק ולהדביק.

חלק רביעי – בניית אפליקציה אמיתית. ניתוב, מצב שחורג מקומפוננטה אחת, דירקטיבות ופייפים מותאמים אישית, וביצועים.

כאן Compass גדל מ"עמוד אחד שעובד" ל"כמה עמודים שנשארים מהירים".

חלק חמישי – הפקה. בדיקות, רינדור בצד השרת (SSR), פריסה, והמערכת האקולוגית שאליה אתה מצטרף. כשתסיים את חלק חמישי, יהיה לך מופע של Compass באינטרנט, ותדע מה לחפש הלאה.

נספחים. מדריך התקנה למחשב, רשימת בדיקות ל-debugging, וסיור קצר באידיומים ישנים של Angular שתפגוש בקוד קיים. קרא אותם כשתצטרך; הם חומר עזר, לא רצף.

איך לקרוא את הספר

קרא אותו ליד המקלדת. לכל פרק יש קוד שכדאי להקליד – לא להדביק. ההקלדה עצמה עובדת: היא מאיטה אותך מספיק כדי לראות מה השתנה, ומאלצת אותך לשים לב לתחביר שהעתקה הייתה מחליקה מעליו. כשפרק מסתיים בסעיף תרגילים, עשה אותם לפני שתתחיל את הפרק הבא. הם לא רשות במובן שהפרק הבא מניח שעשית אותם; הם רשות במובן שאם תדלג עליהם, תאבד את הזיכרון השרירי שהספר מנסה לבנות.

הודעות מודגשות מופיעות לאורך הספר:

הערה. צד-סוגריים שימושי – הקשר מעמיק, מושג קשור, או פיסת היסטוריה.

שים לב. טעות נפוצה או התנהגות עדינה שתחסוך לך אחר צהריים בהמשך.

נסה זאת. ניסוי של דקה לשינוי קוד וראייה של התוצאה. עשה את אלה; הם זולים.

מה צריך

- מחשב נייד או שולחני עם macOS, Windows או Linux.
- כ-5GB של שטח דיסק פנוי (בעיקר עבור `node_modules`, שנפגוש בפרק 4).
- חיבור אינטרנט יציב בזמן העבודה על חלקים ראשון וחמישי; שאר הספר עובד ללא רשת.
- בערך 30 עד 50 שעות פרוסות על פני מספר ישיבות שנוח לך. יש קוראים שסיימו בשני סופי שבוע; יש שפרסו על פני שלושה חודשים. הספר לא אכפת לו.

אתה לא צריך ניסיון קודם ב-Vue, React, Angular או כל פריימוורק אחר. אתה לא צריך לדעת TypeScript; פרק 2 מלמד מספיק ממנו. אתה לא צריך לדעת איך שרתים עובדים; כשנצטרך שרת בחלק שלישי, נרים אותו יחד בכמה שורות של קוד.

מילה על גרסאות של Angular

Angular משחררת גרסה ראשית חדשה בערך כל שישה חודשים, אבל הרעיונות בספר הזה – קומפוננטות, סיגנלים, הזרקת תלויות, ניתוב, טפסים – יציבים ומרכזיים כבר שנים ויישארו כאלה. התחביר הספציפי מיועד ל-Angular 18 ואילך, שהיא הגרסה הראשונה שבה מבנה הזרימה החדש (`@if`, `@for`, `@switch`), סיגנלים, וקומפוננטות עצמאיות הם ברירת המחדל המומלצת. אם אתה קורא את זה כש-Angular נמצאת בגרסה 22 עד אז, רוב הקוד עדיין ירוץ בלי שינוי; מה שכן ישתנה יהיה שינוי שם, לא שינוי חשיבה.

הפוך את הדף. פרק 1 קצר – בלי קוד, רק מפה של איפה אנחנו ולאן אנחנו הולכים.

פרק 1. מהי Angular, ומתי כדאי לפנות אליה

בפרק הזה אין קוד. הוא מפה. לפני שתתקין דבר, יעזור לך לדעת מה אתה עומד להתקין, איזו בעיה זה פותר, ואיזה סוג של אפליקציה אתה עומד לבנות. קרא אותו פעם אחת, לאט. כל מה שיבוא מפרק 2 ואילך מניח שהמפה הזאת בראש שלך.

הצורה של אפליקציית ווב

דמיין, לרגע, אתר שאתה משתמש בו לעתים קרובות. אולי זה יומן, רשימת משימות, שירות סטרימינג של מוזיקה, או תיבת דואר. עכשיו דמיין מה הוא עושה כשאתה מתקשר איתו.

אתה לוחץ על הודעה והטקסט המלא שלה מופיע בחלונית מימין. אתה מקליד שאילתת חיפוש ותוצאות מתסננות תוך כדי הקלדה, בלי שהעמוד נטען מחדש. אתה מסמן משימה כבוצעה והיא עוברת לרשימת "הושלמו"; משתמש אחר, ביבשת אחרת, רואה את השינוי שנייה לאחר מכן. אתה גורר אירוע ביומן שלך והוא מצטרף לזמן חדש.

שום דבר מזה אינו עמוד. בשנים הראשונות של הווב, כל אחת מהאינטראקציות האלה הייתה מחייבת מסע הלוך ושוב לשרת: לחיצה, המתנה, צפייה בעמוד מתלבן, המתנה, ראיית עמוד חדש. אפליקציית ווב מודרנית עושה משהו שונה. היא נטענת פעם אחת, ואז שומרת

תמונה חיה של מה שהמשתמש מסתכל עליו, מעדכנת חלקים מהמסך כשמידע חדש מגיע וכשהמשתמש פועל. כשהיא צריכה מידע או רוצה לשמור משהו, היא מדברת עם השרת בשקט ברקע, בלי לשבש את התמונה.

לשמור על התמונה החיה מעודכנת קשה יותר ממה שזה נשמע. כל חלק על המסך תלוי בפיסת מידע כלשהי – מספר ההודעות שלא נקראו בסרגל הצד תלוי ברשימת ההודעות, מצב הכפתור "השב" תלוי בכך שהודעה נבחרה, הצבע של משימה תלוי אם היא באיחור. כשכל פיסת מידע משתנה, כל חלק של המסך שתלוי בה חייב להשתנות איתה. אם היית מנסה לכתוב את זה ידנית – כל כפתור שצריך להיכנס לפעולה ולצאת ממנה, כל מונה שצריך לגדול, כל רשימה שצריכה לרנדר מחדש – היית מבלה את רוב הזמן שלך בהדבקת דברים במקום לבנות משהו חדש. גם היית מכניס באגים בלי הפסקה: שוכח לעדכן מונה, שוכח להשבית כפתור, שוכח להסיר פריט ישן מרשימה.

פריימוורק הוא תוכנה שמטפלת ברוב ההדבקה הזאת בשבילך. Angular היא פריימוורק.

מה "פריימוורק" באמת אומר

אם כתבת תוכנית מחשב בעבר – אפילו קטנה – כתבת את הקוד שלך ובאיזשהו שלב ייבאת את הקוד של מישהו אחר כדי לעשות עבודה שלא רצית לעשות בעצמך. הקוד של אותו מישהו נקרא **ספרייה**. הקוד שלך אחראי; כשהוא צריך עזרה, הוא קורא לספרייה.

פריימוורק הוא ההפך. בפריימוורק, הקוד של הפריימוורק אחראי; הקוד שלך ממלא את החלקים שהפריימוורק לא יודע. משל שימושי: ספרייה היא פטיש, ואתה מחליט מתי להניף אותו. פריימוורק הוא סדנה, ואתה מחליט מה לבנות בתוך הסדנה – אבל הסדנה כבר בחרה את פריסת שולחן העבודה, את מיקום השקעים, ואת ארונות הכלים על הקיר. אתה מרוויח הרבה: הסדנה מאורגנת, כלים סטנדרטיים תמיד במקום שאתה מצפה שיהיו, ואנשים אחרים שעבדו באותו סוג של סדנה יכולים להתמצא בקוד שלך מייד. אתה גם מוותר על משהו: אם אתה רוצה לעקור את שולחן העבודה ולשים את ארונות הכלים על התקרה, הסדנה תילחם בך.

הבחירה הזאת היא הבחירה המרכזית שאתה עושה כשאתה בוחר פריימוורק. בתמורה לווייתור על גמישות מסוימת, אתה מקבל עקביות, קהילה, והרבה החלטות שאינך צריך יותר לעשות בעצמך. Angular נמצאת בצד ה"בעל דעה מוצקה" של הספקטרום. יש לה דעות על איך לארגן קוד, איך קומפוננטות מדברות זו עם זו, איך לטפל בתופעות לוואי, איך לכתוב בדיקות, ואיך לבנות להפקה. חלק מהחדשים בעניין מוצאים את זה מחניק בימים הראשונים ומשחרר לשארית הקריירה שלהם.

אז – מהי Angular?

Angular היא פריימוורק, שנעשה על ידי Google, לבניית אפליקציות שרצות בדפדפן ווב. במשפט אחד: **אתה מתאר את המסכים של האפליקציה שלך כעץ של חתיכות קטנות ועצמאיות שנקראות קומפוננטות, ו-Angular דואגת להפוך את התיאור הזה לפיקסלים,**

לשמור על הפיקסלים בסנכרון עם הנתונים שלך, ולשלוח את כל העסק לדפדפן.

בואו נפרק את המשפט הזה חלק אחר חלק, כי לכל שם עצם בו יהיה פרק משלו בהמשך.

עץ של קומפוננטות. לא תבנה את Compass כקובץ ענק אחד. תבנה אותו כעץ של חלקים קטנים: מעטפת ברמה העליונה, בתוכה סרגל צד ואזור מרכזי, בתוך האזור המרכזי כותרת ורשימה, בתוך הרשימה שורות רבות של משימות, וכן הלאה. כל חלק – כל קומפוננטה – אחראי לדבר אחד: איך הוא נראה, איך הוא מתנהג כשמתקשרים איתו, ואיך הוא חושף את עצמו לחלקים שסביבו. קומפוננטות בעץ מדברות עם ההורים והילדים שלהן דרך חיבורים מוגדרים היטב וחד-כיווניים. Angular קוראת לחיבורים האלה inputs ו-outputs; תפגוש אותם כראוי בפרק 8.

עצמאיות. ה-HTML, ה-CSS וההתנהגות של הקומפוננטה חיים ביחד. זהו היפוך מכוון של המודל הישן של פיתוח ווב, שבו לפרויקט הייתה תיקייה של קבצי HTML, תיקייה של קבצי CSS, ותיקייה של קבצי JavaScript, וכל פיצ'ר יחיד נגע בקובץ אחד בכל אחת. הטיעון של Angular, שנולד מעשור של צפייה באנשים בונים אפליקציות גדולות, הוא שפיצ'רים לעתים רחוקות מכבדים את הגבולות האלה – הסימון של כפתור, הסגנון שלו, ומטפל הלחיצה שלו הם פיצ'ר אחד – ושמועיל יותר לשמור אותם יחד ולהפריד בין פיצ'רים במקום.

Angular הופכת את התיאור שלך לפיקסלים. אתה מתאר איך המסך צריך להיראות בכל רגע נתון, בהינתן נתונים מסוימים. Angular מבינה מה בעצם לשים על המסך, ומה לשנות כשהנתונים משתנים. זה

עדין. בווּב הישן, כתבת קוד שאמר "כשהמשתמש לוחץ 'השב', הסר את כפתור ההשבה, הוסף כפתור שלה, שים textarea באמצע, הצב עליו את המיקוד". ב-Angular, אתה כותב: "כפתור ההשבה גלוי כשאין טיוטה; כפתור השליחה גלוי כשיש טיוטה; תוכן ה-textarea קשור לטקסט של הטיוטה". אתה מתאר את המצבים, לא את המעברים. Angular עושה את המעברים.

Angular שומרת על הפיקסלים בסנכרון עם הנתונים שלך. כי התיאור שלך אומר איזה חלקים של המסך תלויים באיזה חלקים של הנתונים, Angular יודעת איך לעדכן אותם כשהנתונים משתנים. Angular מודרנית עושה את זה עם מנגנון שנקרא סיגנלים, שתפגוש בפרק 7. סיגנל הוא פיסת נתונים שיודעת מי מסתכל עליה; כשהנתונים משתנים, כל מי שמסתכל עליהם מקבל התראה, ורק החלקים של המסך שקוראים אותם מרונדרים מחדש. זה נשמע כמו נחמדות ומתגלה כמהפכני: זה אומר שהאפליקציה שלך נשארת מהירה גם כשהיא גדלה, כי הפריימוורק אף פעם לא צריך לבדוק שוב את כל המסך אחרי כל שינוי.

Angular שולחת את כל העסק לדפדפן. אפליקציות ווב מודרניות עוברות שלב בנייה שהופך את קוד המקור שלך (הרבה קבצים בפורמט חיבור ידידותי) למספר קטן של קבצים, ממוקטמים בכבדות, מוכנים להישלח לדפדפן. Angular מגיעה עם כלי בנייה שמטפל בזה עבורך. בפרק 4 תריץ את `ng build` בפעם הראשונה ותראה מזה-בייטים של קוד המקור שלך דחוסים למאות בודדות של קילו-בייטים של פלט מוכן להפקה.

מה הופך את Angular ל Angular

Angular היא לא הפריימוורק היחיד מסוגה. אם שמעת את השם Angular, כנראה שמעת גם את השמות של כמה אחרים. הספר הזה לא ישווה בין כולם לעומק – אתה לא צריך את ההשוואה הזאת כדי ללמוד Angular יותר משאתה צריך להשוות בין ספרדית לאיטלקית כדי ללמוד ספרדית – אבל הוגן לתת שם לתכונות שמבדילות את Angular מעמיתיה.

בעלת דעה מוצקה וכוללת סוללות. Angular מגיעה עם ניתוב, טפסים, HTTP, אנימציה, בדיקות, בינאום, ורינדור בצד השרת כבר בקופסה, כולם מעוצבים לעבוד יחד. פריימוורקים שנוקטים בגישה ההפוכה – שולחים ליבה קטנה ונותנים לך לבחור ספריות לכל השאר – נותנים לך יותר גמישות ומאלצים אותך לקבל יותר החלטות. ההימור של Angular הוא שלרוב ההחלטות האלה יש תשובות נכונות רוב הזמן, והיא עושה אותן עבורך כדי שתוכל להשקיע את תשומת הלב שלך באפליקציה שלך במקום זה.

TypeScript תחילה. Angular כתובה ב-TypeScript, וכל אפליקציית Angular נכתבת ב-TypeScript. TypeScript היא סופר-סט של JavaScript: זה JavaScript עם טיפוסים. אם זה נשמע מפחיד, אל דאגה – פרק 2 מלמד מספיק TypeScript כדי להיות פרודוקטיבי, והכלים של Angular גורמים לטיפוסים לעזור לך יותר משהם מפריעים. התמורה אמיתית: כשאתה משנה שם של שדה, העורך שלך משנה את שמו בכל השימושים שלו על פני האפליקציה כולה; כשאתה שובר את החוזה של קומפוננטה עם ההורה שלה, העורך שלך אומר לך לפני שאתה מרענן

את הדפדפן. באפליקציה גדולה, זה ההבדל בין קוד שאתה יכול להחזיק בראש לבין קוד שאתה נלחם בו.

הזרקת תלויות. ל-Angular יש מערכת מובנית לקומפוננטה אחת לומר "אני צריכה משהו שעושה X" ולקבל דבר מתאים בלי שתצטרך לדעת איך הוא נבנה. זה נשמע מופשט עכשיו; זה ירגיש מובן מאליו אחרי פרק 9. זו הסיבה הבודדת הגדולה ביותר לכך שאפליקציות Angular נוטות להיות בדיקות, כי "תן לי גרסה מזויפת של הדבר שמדבר עם השרת" מתגלה כשורת קוד אחת.

סיגנלים כפרימיטיב הריאקטיבי. רוב ההיסטוריה של Angular, המודל הריאקטיבי שלה נבנה על ספרייה שנקראת RxJS, שמייצגת שינויים לאורך זמן בזרמים נצפים. RxJS חזקה ועדיין חשובה – תפגוש אותה בפרק 11 – אבל צוק הלמידה היה תלול והטקסים היו כבדים למצב יומיומי. גרסאות אחרונות של Angular הציגו סיגנלים, פרימיטיב קל יותר שמרגיש כמו משתנים רגילים אבל מעדכן את המסך אוטומטית כשהם משתנים. סיגנלים הם איך שתנהל את רוב המצב בספר הזה; RxJS הוא איך שתטפל בזרמים שנפרסים במהותם לאורך זמן, כמו תגובות HTTP וזרמי אירועים.

CLI מונחה קונבנציות. Angular מגיעה עם כלי שורת פקודה, ng, שמייצר תבניות של פרויקטים, יוצר קומפוננטות ושירותים חדשים, מריץ את האפליקציה שלך, מריץ את הבדיקות שלך, ובונה להפקה. זה הכלי שמסיר את רוב הטקסים של ההקמה שפעם היו שער לפיתוח ווב. בפרק 4 תשתמש בו כדי ליצור את Compass בפקודה אחת.

מתי Angular היא בחירה טובה – ומתי לא

Angular היא לא הבחירה הנכונה לכל פרויקט בווב, וחלק מהלמידה של הפריימוורק הוא לדעת איפה הוא זורח.

Angular היא במיטבה כשהאפליקציה היא כזאת שאתה מצפה לעבוד עליה לאורך זמן, עם צוות, עם מורכבות עסקית אמיתית: דשבורדים, פאנלים של מנהלים, כלים פנימיים, פורטלים ללקוחות, אפליקציות בנקאות ובריאות ומסחר אלקטרוני שיצמחו פיצ'רים ברבעון במשך שנים. בהקשרים האלה, המבנה בעל הדעה של Angular, יסוד ה-TypeScript, וסיפור הבדיקות משתלמים פי כמה. מהנדסים חדשים יכולים להצטרף ולהתמצא; refactor בטוחים כי הקומפיילר תופס שבירות של חוזים; ניתן להוסיף פיצ'רים בלי לשבור בטעות שלושה פיצ'רים אחרים.

Angular היא כלי כבד יותר ממה שאתה צריך לעמוד שיווקי בודד, אתר נחיתה, בלוג, או הדגמה אינטראקטיבית חד-פעמית. עבור אלה, HTML פשוט – או מחולל אתרים סטטי קל משקל – יתחיל אותך מהר יותר, ישלח פחות קוד למשתמשים שלך, ויהיה קל יותר להעביר הלאה. לבחור ב-Angular לאתר שיווקי של חמישה עמודים זה כמו לשכור מחסן כדי להכין כריך. אתה יכול לעשות את זה; זה יעבוד; לא כדאי לך.

Angular היא לא הבחירה הטובה ביותר למנועי משחקים, ויזואליזציות כבדות, או אפליקציות שהמשטח העיקרי שלהן הוא canvas או סצנת WebGL. Angular לא יודעת ולא אכפת לה מה נראה פיקסל של ה-canvas שלך; היית משתמש בה רק לממשק שסביב

ה-canvas, וגם אז פריימוורק קל יותר או קוד רגיל אולי היה מתאים יותר.

Compass, האפליקציה שאתה עומד לבנות, יושבת במרכז מקום הכיף של Angular: כמה מסכים, נתונים אמיתיים, שרת מתמיד, טפסים, אימות, ורשימת פיז'רים שתגדל לאורך הפרקים. הוא מייצג את סוג האפליקציה שצוותי Angular בונים כל יום.

מה Compass יהיה, בסוף

עד לפרק האחרון, Compass יהיה אפליקציה אמיתית, אם גם קטנה. הנה תצוגה מקדימה כדי שתדע לאן הדרך מובילה.

תפתח את Compass בדפדפן שלך ותראה מסך התחברות. תתחבר עם כתובת מייל; קישור אישור יגיע; תלחץ עליו, ו-Compass יזכור אותך במכשיר הזה. תגיע לדשבורד שמראה את המשימות של היום ואת ההרגלים שאתה עוקב אחריהם השבוע.

תוסיף משימה בלחיצה על כפתור, מילוי טופס, ולחיצה על שמור. היא תופיע מיד. אם האינטרנט שלך כבוי, היא עדיין תופיע מיד, מסומנת באיקון קטן שאומר "עדיין לא סונכרן", וכשהאינטרנט יחזור היא תישמר בשקט ברקע. תגרור משימות בין "היום", "השבוע הזה", ו"מאוחר יותר". תסמן הרגלים כבוצעו בלחיצה; מונה רצפים ליד כל הרגל יתעדכן.

תנווט לעמוד סטטיסטיקות ותראה תרשים של השלמות ההרגלים שלך על פני שלושים הימים האחרונים. תלחץ על הרגל כדי לראות את

ההיסטוריה שלו. תבקר בעמוד הגדרות שבו תוכל לשנות את שם התצוגה שלך ולבחור ערכת נושא.

כל אחד מהפיצ'רים האלה הוא מקום בספר. מסך ההתחברות ואישור המייל חיים בפרקי הניתוב והטפסים. ההתנהגות של offline-first חיה בפרקי ה-HTTP והמצב. התרשים חי בפרק הקומפוננטות והתבניות, ושוב בפרק הביצועים כשנעשה אותו מהיר מספיק כדי לא להיות משנה בטלפון. הבדיקות שתכתוב לכל זה חיות בחלק חמישי.

תסיים את הספר עם אפליקציה שקטנה מספיק כדי להחזיק בראש ועשירה מספיק כדי להצדיק כל רעיון ש-Angular מציעה.

מה בא הלאה

שלושת הפרקים הבאים מכינים אותך להקליד `ng new compass`. הם יוצאי דופן עבור ספר "בונים אפליקציה" כי הם מכילים כמעט אפס Angular. זה במכוון. Angular יושבת על גבי TypeScript, TypeScript יושבת על גבי JavaScript מודרני, ושניהם יושבים על גבי סט של כלים – מנהל חבילות, סביבת ריצה, עורך. דילוג עליהם, כמו שהרבה ספרים עושים, מייצר קוראים שיכולים להעתיק דוגמאות של Angular אבל שקופאים בפעם הראשונה שהודעת שגיאה מזכירה מונח שהספר מעולם לא הגדיר.

פרק 2 מכסה את TypeScript. לא את כולו – TypeScript הוא שפה גדולה עם סט טיפוסים אפשריים גדול עוד יותר – רק את החלקים שבאמת תפגוש כשתכתוב Angular. פרק 3 מכסה את התכונות של JavaScript המודרני ש-Angular מסתמכת עליהן: `destructuring`, `spread`,

פונקציות חץ, מודולים, promises ו- `await / async`. פרק 4 יעביר אותך דרך התקנת Node והתקנת Angular CLI, הרצת `ng new` `compass`, וסיור איטי על כל קובץ שה-CLI מייצר כדי שכלום בפרויקט החדש שלך לא ייראה מסתורי.

בסוף חלק ראשון, תהיה לך אפליקציית Angular רצה על המסך שלך ותבין כל חלק ממה שיצר אותה. אז, בחלק שני, נתחיל לבנות.

תרגילים

1. פתח אפליקציית ווב שאתה משתמש בה לעתים קרובות – לקוח מייל, שירות מוזיקה, כלי לניהול פרויקטים, כל דבר עם ממשק עשיר. שים לב מה קורה כשאתה לוחץ מסביב. נסה לזהות שלוש אינטראקציות שהיו דורשות טעינה מלאה של עמוד באתר בסגנון ישן ובמקום זה מעדכנות רק פיסה של המסך. רשום אותן.

2. במילים שלך, בשלושה או ארבעה משפטים, תאר את ההבדל בין ספרייה לפריימוורק למישהו שמעולם לא כתב קוד. אל תשתמש במילים ספרייה או פריימוורק בתשובה שלך. אם אתה יכול להסביר את החליפין בלעדיהן, אתה מבין אותם.

3. חשוב על רעיון אפליקציה שאולי תרצה לבנות יום אחד. הוא לא חייב להיות מקורי או טוב; קופסת מתכונים אישית, יומן תרגול מוזיקה, או תזכורת ליום הולדת זה בסדר. שאל את עצמך: האם האפליקציה הזאת נמצאת במקום הכיף של Angular, או שמשהו קל יותר ישירות אותה יותר טוב? רשום

את התשובה שלך ואת ההיגיון שלך. בעוד כמה חודשים,
חזור וראה אם אתה עדיין מסכים.